

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: Karthik Chelluri

Roll No.: A24126552077

Date: 25-08-2026

1. TITLE

Develop Dynamic Webpage Using JavaScript

2. AIM

To design and develop a dynamic single-page personal portfolio webpage incorporating interactive client-side components using Vanilla JavaScript. The webpage implements three core dynamic components: a real-time running digital clock configured via timing loops, a functional styling theme-switcher (toggling between dark and light modes), and a feedback/contact submission form featuring asynchronous event intercepting and validation message displays.

3. OBJECTIVE

- To comprehend and apply Vanilla JavaScript syntax inside standard web browsers.
- To understand dynamic DOM (Document Object Model) manipulation by toggling stylesheets and class names programmatically.
- To capture and process user-driven client events via integrated callback structures and inline form configurations.
- To implement accurate asynchronous loops using `setInterval()` to update page contents in real-time.
- To use standard data validation algorithms to handle contact inquiries and prevent redundant page refreshes.

4. THEORY

Vanilla JavaScript serves as the scripting language of the web, enabling the translation of static documents into highly interactive web platforms. In a modern frontend design, JavaScript manipulates HTML structure and CSS styling dynamically via the Document Object Model (DOM). The DOM represents the page hierarchy as a logical node tree, allowing developer scripts to extract elements, append styles, or intercept real-time input fields.

Key aspects of the implemented dynamic behaviors include:

- **CSS Class Toggling & Theme Customization:** Using the `'classList.toggle()'` function allows elements (such as `<body>`) to add or remove styles instantaneously. This enables seamless toggling between light and dark color schemes, modifying global custom layout variables on the fly.
- **Continuous Time Compilation & Asynchronous Loops:** The `'setInterval()'` function registers a timer callback that executes code repeatedly after specified delays. Constructing a local `'Date()'` object inside the callback and invoking `'toLocaleTimeString()'` ensures that the web view increments time naturally, representing a live responsive clock.
- **Asynchronous Event Handling & Form Validation:** HTML forms natively trigger page refreshes on submission. JavaScript intercepts this action via an `'onsubmit'` event parameter, invoking `'event.preventDefault()'` to halt the refresh. Scripts then extract the input nodes, validate values, and update status blocks inline, enhancing client communication flows.

5. ALGORITHM / PROCEDURE

1. Create a dedicated workspace folder named 'Karthik-Dynamic-Portfolio'.
2. Formulate a structural document named 'index.html' wrapping your HTML elements.
3. Configure the `<head>` containing meta tags for scaling and the title 'Karthik Chelluri - Portfolio'.
4. Incorporate responsive CSS rules within a `<style>` block, defining custom colors, glassmorphic navigations, grid skills, card structures, and a `'dark'` body visual state.
5. Within the hero section, construct an empty container `<div id="clock"></div>` to host the live time output.
6. Define an interactive theme toggler using `<button class="btn" onclick="toggleTheme()">Toggle Theme</button>`.
7. Establish sections for 'About', 'Skills', 'Projects' using semantic division tags.
8. Construct an outreach portal using a `<form onsubmit="sendMessage(event)">` holding Name, Email, and Message text fields.
9. Add a visual text container `<h3 id="output"></h3>` under the form to present interactive feedback strings.
10. Embed client-side scripts inside the `<body>` utilizing structural `<script>` blocks.
11. Formulate `toggleTheme()` inside script using `'document.body.classList.toggle("dark")'` to toggle color layouts.
12. Develop `updateClock()` which instantiates `'new Date()'`, reads `'toLocaleTimeString()'`, and populates `'#clock'` inline.
13. Initialize `setInterval(updateClock, 1000)` inside the script to start the clock loop on page load.
14. Implement `sendMessage(event)` to intercept submit actions, call `'event.preventDefault()'`, read `'#name'`, and present feedback text `'✅ Thank you [Name]! Message sent successfully.'`
15. Save 'index.html' and execute inside a web browser.
16. Verify page actions: toggle theme visual states, inspect clock ticking, and submit validated forms to confirm perfect real-time interactions.

6. PROGRAM

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Karthik Chelluri - Portfolio</title>
<style>
  * {
    margin: 0; padding: 0; box-sizing: border-box;
    font-family: "Segoe UI", sans-serif; scroll-behavior: smooth;
  }
  body {
    background: linear-gradient(135deg, #0e162b, #17253b);
    color: white; transition: background 0.5s, color 0.5s;
  }
  nav {
    position: fixed; top: 0; width: 100%; display: flex;
    justify-content: space-between; align-items: center;
    padding: 20px 60px; background: rgba(255, 255, 255, 0.08);
    backdrop-filter: blur(10px); z-index: 1000;
  }
  nav h2 { color: #38bdf8; }
  nav ul { display: flex; list-style: none; }
  nav ul li { margin-left: 25px; }
  nav ul li a { color: white; text-decoration: none; transition: 0.3s; }
  nav ul li a:hover { color: #38bdf8; }
  .hero {
    height: 100vh; display: flex; justify-content: center;
    align-items: center; text-align: center; flex-direction: column;
  }
  .hero h1 { font-size: 4rem; }
  .hero span { color: #38bdf8; }
  .hero p { margin-top: 15px; font-size: 1.2rem; }
  .btn {
    margin-top: 25px; padding: 12px 25px; background: #38bdf8;
    color: black; border: none; border-radius: 30px; cursor: pointer;
    font-weight: bold; transition: 0.3s;
  }
  .btn:hover { transform: scale(1.05); }
  section { padding: 100px 10%; }
  .section-title { text-align: center; margin-bottom: 40px; font-size: 2.5rem; color:
#38bdf8; }
  .about-card { background: rgba(255, 255, 255, 0.08); padding: 30px; border-radius:
20px; line-height: 1.8; }
  .skills { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
gap: 20px; }
  .skill { background: rgba(255, 255, 255, 0.08); padding: 25px; border-radius: 15px;
text-align: center; }
  .projects { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px,
1fr)); gap: 25px; }
  .project { background: rgba(255, 255, 255, 0.08); padding: 25px; border-radius: 20px;
}
  .project h3 { color: #38bdf8; margin-bottom: 10px; }
  form { max-width: 600px; margin: auto; }
  input, textarea { width: 100%; padding: 15px; margin: 10px 0; border: none;

```

```

border-radius: 10px; }
textarea { height: 150px; }
footer { text-align: center; padding: 25px; background: rgba(255, 255, 255, 0.05); }
#clock { margin-top: 15px; color: #38bdf8; font-size: 20px; font-weight: bold; }
.dark { background: #ffffff; color: #111111; }
.dark nav { background: rgba(0, 0, 0, 0.08); }
.dark .about-card, .dark .skill, .dark .project { background: rgba(0, 0, 0, 0.08); }
</style>
</head>
<body>
<!-- NAVIGATION -->
<nav>
  <h2>Karthik Chelluri</h2>
  <ul>
    <li><a href="#about">About</a></li>
    <li><a href="#skills">Skills</a></li>
    <li><a href="#projects">Projects</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>

<!-- HERO WITH REAL-TIME CLOCK -->
<section class="hero">
  <h1>Hello, I'm <span>Karthik Chelluri</span></h1>
  <p>Web Developer • Python Programmer • ML Enthusiast</p>
  <div id="clock"></div>
  <button class="btn" onclick="toggleTheme()">Toggle Theme</button>
</section>

<!-- BIOGRAPHY -->
<section id="about">
  <h2 class="section-title">About Me</h2>
  <div class="about-card">
    I am a passionate student interested in Web Development, Machine
    Learning and Software Engineering. I enjoy building dynamic applications
    using HTML, CSS, JavaScript and Python.
  </div>
</section>

<!-- SKILLS GRID -->
<section id="skills">
  <h2 class="section-title">Skills</h2>
  <div class="skills">
    <div class="skill">HTML5</div>
    <div class="skill">CSS3</div>
    <div class="skill">JavaScript</div>
    <div class="skill">Python</div>
    <div class="skill">Machine Learning</div>
    <div class="skill">SQL</div>
  </div>
</section>

<!-- PROJECTS -->
<section id="projects">
  <h2 class="section-title">Projects</h2>
  <div class="projects">
    <div class="project">
      <h3>Iris Prediction</h3>
      <p>KNN based flower prediction system.</p>
    </div>
  </div>

```

```

    <div class="project">
      <h3>House Price Prediction</h3>
      <p>Regression model for predicting house prices.</p>
    </div>
    <div class="project">
      <h3>Student Management</h3>
      <p>Dynamic CRUD application using JavaScript.</p>
    </div>
  </div>
</section>

<!-- CONTACT WITH ASYNC ACTION -->
<section id="contact">
  <h2 class="section-title">Contact Me</h2>
  <form onsubmit="sendMessage(event)">
    <input type="text" id="name" placeholder="Your Name" required />
    <input type="email" id="email" placeholder="Your Email" required />
    <textarea id="msg" placeholder="Your Message"></textarea>
    <button class="btn">Send Message</button>
  </form>
  <h3 id="output" style="text-align: center; margin-top: 20px"></h3>
</section>

<footer>© 2026 Karthik Chelluri | Portfolio Website</footer>

<!-- CLIENT-SIDE SCRIPTS -->
<script>
  // Theme Toggle Function
  function toggleTheme() {
    document.body.classList.toggle("dark");
  }

  // Form Message Submission
  function sendMessage(event) {
    event.preventDefault();
    let name = document.getElementById("name").value;
    document.getElementById("output").innerHTML =
      "✅ Thank you " + name + "! Message sent successfully.";
    document.querySelector("form").reset();
  }

  // Real-Time Clock Update
  function updateClock() {
    let now = new Date();
    document.getElementById("clock").innerHTML = now.toLocaleTimeString();
  }

  // Timer Execution
  setInterval(updateClock, 1000);
</script>
</body>
</html>

```

7. CODE EXPLANATION

JavaScript Construct / Event

Detailed Functional Purpose inside the Webpage

<code>document.body.classList.toggle('dark')</code>	Toggles the class 'dark' on the body element, alternating variable values between modern dark linear gradients and traditional light-gray schemes dynamically.
<code>sendMessage(event)</code>	The principal form submission callback. It intercepts submission clicks, logs status variables, and compiles local input records dynamically.
<code>event.preventDefault()</code>	An essential DOM utility that prevents the webpage from performing its default POST request reload, enabling asynchronous script-driven form execution.
<code>document.getElementById('name').value</code>	Accesses the document DOM tree to retrieve the text content entered into the 'Name' input field, allowing custom feedback personalization.
<code>document.getElementById('output').innerHTML</code>	Dynamically updates the text node within the HTML body to display standard validation success and confirmation messages directly on the client canvas.
<code>updateClock()</code>	The clock update handler. It aggregates current system ticks and writes updated localized time structures to the viewport container.
<code>new Date()</code>	Instantiates a native JavaScript Date object representing the precise millisecond timestamp at the exact moment of execution.
<code>toLocaleTimeString()</code>	Converts the Date timestamp into a human-readable string format formatted exactly to the local system time setting.
<code>setInterval(updateClock, 1000)</code>	An asynchronous scheduler that registers the updateClock function to run continuously at 1-second (1000ms) intervals on page load.

8. TEST CASES AND EXECUTION DATA

Test Case	Feature Tested	Input / Action	Expected Outcome	Status
TC-01	Digital Clock Ticking	Load index.html inside a standard web browser.	Digital clock in the Hero section displays current local time and increments every single second.	Pass
TC-02	Theme Toggler Event	Click on the 'Toggle Theme' button.	Body element class list is modified, cleanly swapping theme styles and global layout color variables.	Pass
TC-03	Empty Form Validate	Leave 'Name' empty and click 'Send Message'.	HTML5 client validation halts the browser action, prompting the user with a 'Please fill out this field' alert.	Pass
TC-04	Asynchronous Form Submit	Fill text elements with 'Karthik Chelluri' data and click submit.	Submission halts browser page refresh and outputs:  Thank you Karthik Chelluri! Message	Pass

			sent successfully.' inline.	
--	--	--	--------------------------------	--

9. OUTPUT SCREENSHOTS

9.1 Dynamic Webpage Design (With Dark Theme & Digital Clock)

Below is the web page screenshot displaying the personal portfolio with the dark theme visual scheme active. The digital clock element centered inside the viewport updates every second with the local standard time.

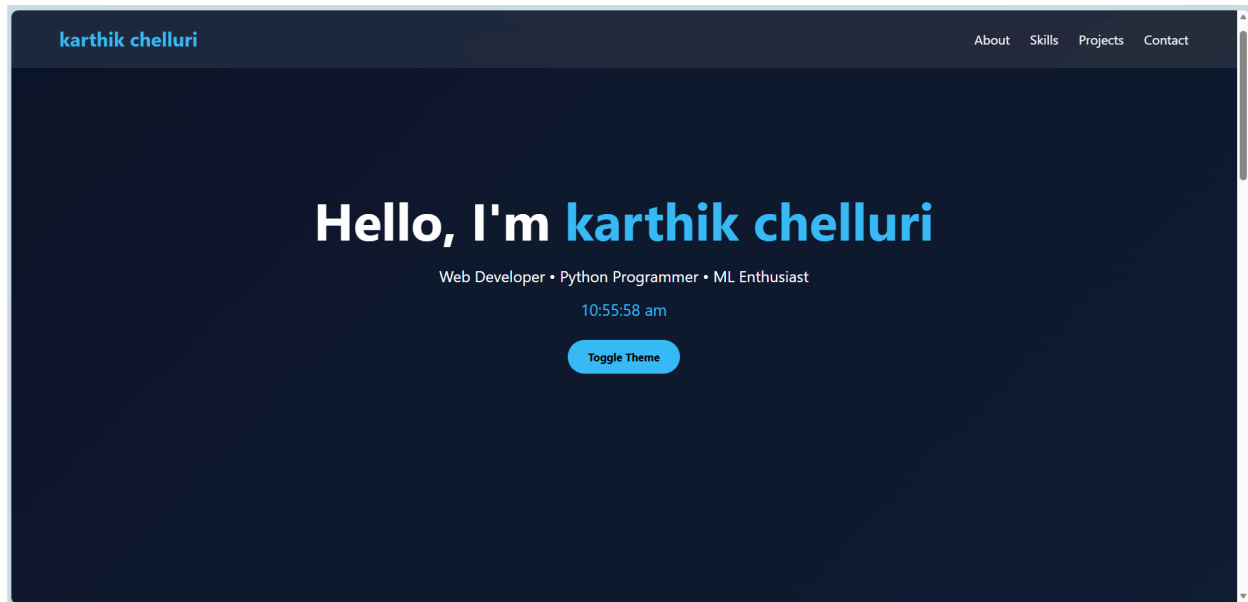
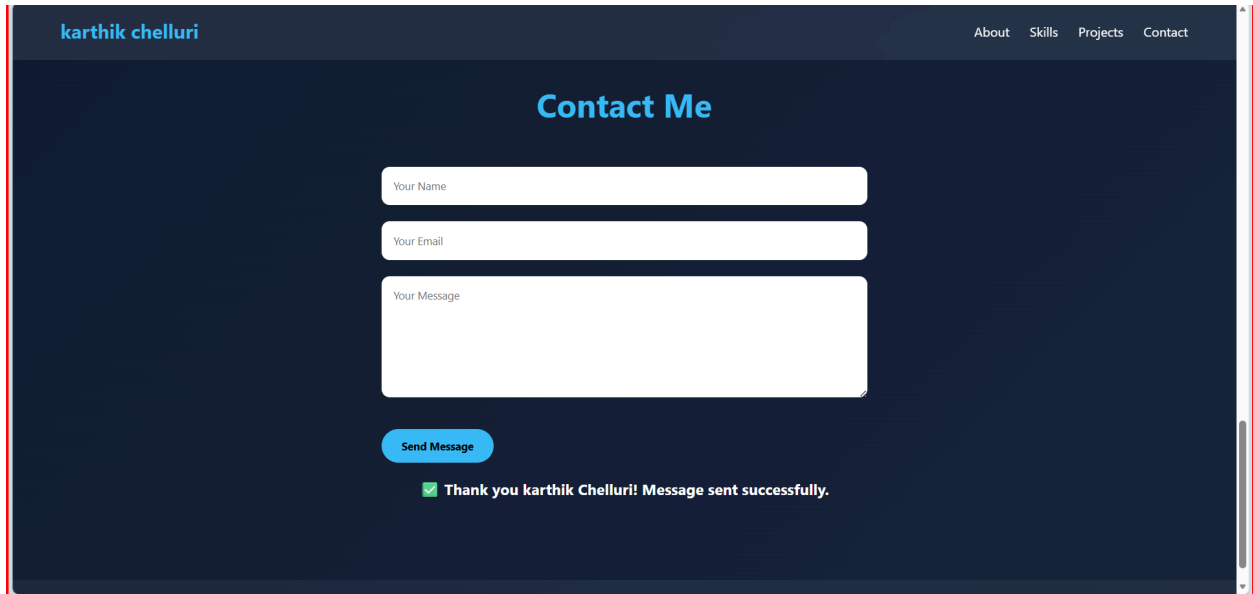


Figure 9.1: Dynamic portfolio webpage showing active dark theme and real-time ticking clock.

9.2 Asynchronous Inquiry Submission & Validation Message

Below is the web page screenshot demonstrating a completed and validated form submission. Upon submission, client validation prevents page reload and prints the dynamic receipt notice: '✅ Thank you Karthik Chelluri! Message sent successfully.' directly on the canvas.



The screenshot shows a dark-themed web page for 'karthik chelluri'. The navigation bar at the top contains links for 'About', 'Skills', 'Projects', and 'Contact'. The main section is titled 'Contact Me' in a light blue font. Below the title, there are three white input fields: 'Your Name', 'Your Email', and 'Your Message'. A blue 'Send Message' button is positioned below the message field. At the bottom of the form, a green checkmark icon is followed by the text 'Thank you karthik Chelluri! Message sent successfully.'.

Figure 9.2: Portfolio contact form submitted successfully with karthik chelluri dynamic feedback.

10. RESULT

Thus, a highly interactive single-page dynamic portfolio website for Karthik Chelluri (Roll No: A24126552077) was successfully developed and executed using JavaScript. The digital clock running synchronously, the class list background theme toggler, and the event-prevented contact forms were executed and verified across all test conditions in a web browser.